BUILD SPECIFICATION

Patent Prior-Art Search Engine

A Hybrid-Retrieval, Reranked, Explainable Prior-Art Discovery System

Domain	Intellectual Property / Legal Tech
System type	Multi-stage hybrid retrieval + LLM reasoning pipeline
Delivery target	Production-deployed, self-tested, resume-ready portfolio system
Document purpose	Hand this file to an IDE coding agent (Cursor, Claude Code, Windsurf, Copilot Workspace) as its build spec
Document version	1.0 — Generated for autonomous phase-by-phase execution

How to use this document

Feed this file (or Section 6 prompts individually) to your IDE agent one phase at a time. Do not let the agent skip a phase's Definition-of-Done gate — each phase is written to be independently verifiable before the next begins.

Table of Contents

0. Executive Summary & Engineering Reality Check

0.1 What this system actually does

This system takes a raw patent claim (independent claim text, e.g. Claim 1 of a granted patent or pending application), decomposes it into atomic technical limitations, and searches a corpus of prior patents and non-patent literature for documents that could invalidate the claim — i.e. anticipate or render obvious one or more of its elements. It runs the search through four progressively smarter retrieval layers and proves, with a measured lift table, that each layer is actually worth the latency and cost it adds.

0.2 The four things a resume reviewer or interviewer will actually probe

- **"Why not just use one embedding model?"** Because dense retrieval will happily return a semantically similar patent while missing the one with the exact chemical formula "$C_9H_8O_4$" or patent number "US-7,123,456-B2" in it. That's the forcing function this whole architecture exists to demonstrate — you must be able to show the failure mode, not just describe it.
- **"How do you know reranking actually helped?"** The lift table (Phase 5) with Precision@k / Recall@k / MRR / nDCG measured per layer, on a held-out labeled set, is the answer. Without labeled ground truth, this project is a demo. With it, it's a real evaluation.
- **"Is this legally reliable?"** No — and the system must say so, in the UI, every time. This is a portfolio engineering project, not a substitute for a registered patent attorney or a professional prior-art search firm. Section 0.4 covers exactly how to phrase this honestly.
- **"Where does your data come from?"** Real, freely available patent full-text and claims data (USPTO PatentsView / Google Patents Public Datasets / EPO OPS), not synthetic data. Section 1 pins the exact sources so the agent doesn't invent a fake corpus.

0.3 Non-negotiable constraint this plan is built around

> **Forcing function (restated from the brief, kept as the system's north star)**
> Pure dense retrieval silently drops exact patent-number and chemical-formula matches. Pure BM25 misses semantic paraphrase of the same technical concept. A missed invalidating document is a flawed legal opinion with real liability. Every retrieval layer in this build (BM25, dense, RRF fusion, cross-encoder rerank, query transform) must earn its place with a measured precision/recall improvement on a labeled eval set — not be included because it "sounds more advanced." If a layer does not measurably help in the Phase 5 lift table, the plan requires you to report that honestly, not hide it.

0.4 Mandatory disclaimer text (ship this, don't skip it)

Because this system touches patent invalidity — a real legal determination with real consequences — every build phase that produces user-facing output (API responses, UI, README, resume description) must carry a visible, unambiguous non-legal-advice disclaimer. This is not a nice-to-have; Phase 4 (UI) and Phase 7 (deployment) each have an explicit Definition-of-Done line requiring it to be present and untestable-away. Suggested exact copy is provided in Section 1.5 and again in the Phase 4 spec so the agent has no excuse to omit it.

0.5 What "production grade" concretely means for this specific plan

The request asks for production-grade, self-testing, self-deploying, error-free delivery. For a solo/portfolio build, "production grade" is defined precisely as the following seven properties, each mapped to a phase below so the agent has an unambiguous target rather than a vague aspiration:

Production-grade property	Owned by phase	Objective, checkable signal
Reproducible from clean clone	Phase 1, 7	`docker compose up` (or documented one-command script) succeeds on a machine with nothing pre-installed but Docker
Correct retrieval behavior	Phase 2, 3	Unit + integration tests pass; each retriever returns expected docs on fixture queries
Measurably better than naive baseline	Phase 5	Lift table shows monotonic or explained precision/recall improvement layer over layer
Fails safely, not silently	Phase 3, 6	Timeouts, empty results, and malformed claims return typed errors, not crashes or empty 200s
Observable in production	Phase 6	Structured logs, request tracing, and a `/health` + `/ready` endpoint exist and are tested
Secured against obvious abuse	Phase 6	Rate limiting, input length caps, and API-key auth are present and tested, not "planned"
Documented for a stranger	Phase 8	A reviewer with zero context can read the README and understand architecture, run it, and see the eval results in under 10 minutes

1. Foundational Decisions (Read Before Coding Starts)

These decisions are made once, up front, so the agent does not re-litigate them mid-build. Treat this section as fixed configuration, not brainstorming.

1.1 Corpus & data sources — pin these exactly

Do not let the coding agent invent a synthetic patent corpus — the entire forcing function of this project (exact-match failure vs semantic-match failure) is only demonstrable on real patent text, because only real claims contain real chemical formulae, real patent-number cross-references, and real paraphrase drift between related filings.

Source	What it provides	Access	Use in this system
Google Patents Public Datasets (BigQuery, `patents-public-data`)	Full-text claims, abstracts, descriptions, citations, CPC codes for 100M+ publications, global coverage	Free, public BigQuery dataset; no key required for read	Primary corpus source — pull a bounded slice (see 1.2) for the vector + BM25 indexes
USPTO PatentsView API	Structured US patent metadata, citations, assignees, claims	Free, API key via self-service signup	Metadata enrichment + citation graph for eval-set construction
USPTO Patent Public Search / Bulk Data (PatFT/AppFT successor)	Full official claim text, PDFs, prosecution history	Free, bulk XML/JSON downloads	Fallback/cross-check for claim text fidelity

Source	What it provides	Access	Use in this system
EPO Open Patent Services (OPS)	European + global patent full text, legal status	Free tier with registration, rate-limited	Optional: broaden corpus beyond US-only for a stronger portfolio story
Lens.org (Lens Patent corpus)	Patent + scholarly non-patent-literature (NPL) linking	Free for individual/academic use, registration required	Optional Phase 3.5 stretch — add NPL as an invalidating-source type, not just patents

Agent instruction
Confirm current terms of use and rate limits for whichever source is selected before pulling data at scale — these change. Do not scrape any site that disallows it in robots.txt or ToS. If BigQuery access is unavailable in the dev environment, fall back to USPTO bulk XML downloads for a bounded technology area (Section 1.2) rather than inventing placeholder patents.

1.2 Bound the corpus — do not attempt "all patents"

A portfolio project does not need, and should not attempt, full USPTO+WIPO+EPO coverage. Bound the corpus to keep indexing time, cost, and eval-labeling effort sane:

1. Pick 2–3 CPC (Cooperative Patent Classification) subclasses in one technology area you can reason about — e.g. H01M (batteries/electrochemistry) or A61K (pharmaceutical preparations) or G06N (machine learning). A narrower, coherent domain makes the semantic-paraphrase forcing function easier to demonstrate and easier to hand-label.
2. Target corpus size: 50,000–150,000 patent documents (claims + abstract + first ~2 independent claims minimum; full description text optional and can be added later for the HyDE step). This is large enough to make BM25-only and dense-only failure modes real, small enough to index on a single machine.
3. Include documents with intentionally varied surface form for the same underlying concept (continuations, related family members, differently-worded competitor filings) — this is what makes RRF and reranking demonstrably earn their place, versus a corpus where every doc is trivially distinct.

1.3 Ground truth / eval-set construction — the single most important artifact in this project

Why this matters more than any model choice
The brief explicitly demands "precision and recall at k" and a lift table. Without labeled query→relevant-document pairs, none of that is possible — you'd just be asserting the system "feels" better. This eval set is Phase 0 of engineering work, done before any retrieval code, and it is the artifact that turns this from a demo into an evaluated system.

Two realistic ways to build it, in order of preference:

4. **Use real patent citation data as weak labels.** Patents cite prior art during prosecution (the "References Cited" / 892/1449 forms, exposed in PatentsView and Google Patents data). For a held-out set of ~50–100 claims, treat their examiner-cited prior art as the relevant set. This is real-world ground truth, free, and defensible in an interview ("I used examiner citations as weak relevance labels, which is a known, referenced approach in patent-retrieval literature").

5. **Manually label a small gold set.** For 15–25 claims, manually review top-20 candidates from each retrieval layer and hand-label relevant/not-relevant. Time-boxed, but produces the highest-quality signal and is worth doing for at least a subset even if (1) is your primary source, to sanity-check that citation-based labels aren't systematically biased toward one retrieval style.

Target: a held-out eval set of at least 40 claims with labeled relevant documents, kept completely separate from anything used to tune BM25/embedding/rerank parameters, stored as a versioned JSON/CSV fixture in the repo (`/eval/gold_set.json`) so results are reproducible run over run.

1.4 Tech stack — locked-in choices and the reasoning

Layer	Choice	Why (and the honest alternative)
Orchestration	LangChain (LCEL) or LangGraph for the multi-step pipeline	Matches brief's stated stack; LangGraph specifically if the pipeline needs branching/retry logic per stage, which it does (Phase 3)
Vector DB	Qdrant (self-hosted via Docker, or Qdrant Cloud free tier)	Open-source, first-class hybrid search support (dense + sparse in one query), strong filtering for CPC/date. Weaviate is an equally valid substitute — note it in the README as a documented alternative, don't build both
Sparse/lexical retrieval	BM25 via `rank-bm25` for the from-scratch demonstration; consider Qdrant's built-in sparse vectors (SPLADE) as a stretch upgrade in Phase 3.5	`rank-bm25` is simple, inspectable, and directly matches the brief. Note in README that Qdrant-native sparse vectors are a valid production alternative once past the learning/demo goal
Dense embeddings	A strong general or domain embedding model reachable via API or local inference (confirm current best option — e.g. an OpenAI embedding model, Cohere embed, or an open BGE-family model) at build time	Do not hardcode a specific model version here — embedding model availability and quality shifts fast; Phase 2 has the agent verify current options before locking one in
HyDE / query decomposition LLM	Groq (fast open-weight models) for latency-sensitive steps, OpenAI/Anthropic for higher-reasoning decomposition — confirm current model names at build time	Brief specifies Groq or OpenAI; keep this provider-swappable behind one interface (Phase 2) rather than hardcoded, since both model lineups change
Fusion	Reciprocal Rank Fusion (RRF), implemented from scratch (~15 lines), not a library black box	RRF's entire appeal is no tunable weight — implementing it directly makes that property visible and interview-explainable
Reranker	Cohere Rerank API (managed) with a self-hosted BGE cross-encoder as a documented fallback/offline option	Brief specifies either; building the interface to swap between them is itself a good engineering signal (see Phase 3 adapter pattern)

Layer	Choice	Why (and the honest alternative)
Eval harness	Custom precision/recall/MRR/nDCG harness, with RAGAS optionally layered on top for LLM-output-quality metrics (explanation faithfulness) in Phase 5.5	Custom harness is required because RAGAS alone doesn't give retrieval-layer P@k/R@k lift tables — that's bespoke to this project's forcing function
Backend API	FastAPI (Python), async throughout	Natural fit with LangChain/LangGraph Python ecosystem, built-in OpenAPI docs support Phase 8 documentation requirement
Frontend	Next.js + TypeScript, minimal component library (no heavy design system needed)	Optional per Phase 4 — see note that a clean API + Swagger UI can stand alone if time-boxed; frontend adds resume polish, not core engineering signal
Deployment	Docker Compose for full local reproduction; Railway/Render/Fly.io (backend) + Vercel (frontend) for a live demo URL, or a single VM/EC2 with Docker Compose if a live URL isn't needed	Section 7 details a decision tree so cost/effort is scoped to what the person actually needs (resume link vs local-only)

1.5 Standing disclaimer copy (reuse verbatim across UI, README, API responses)

```
This tool assists prior-art research and is NOT a substitute for a registered
patent attorney, patent agent, or professional prior-art search firm.
Results are retrieval-and-ranking outputs from an automated pipeline and
have not been reviewed by a legal professional. Do not rely on this tool's
output, alone, for any filing, licensing, litigation, or invalidity decision.
```

PHASE 0

Repository, Environment & Contracts

Goal
Stand up a clean, reproducible skeleton with the interfaces (data contracts) fixed before any retrieval logic is written, so every later phase builds against a stable shape instead of guessing.

Estimated effort: 0.5–1 day

Tasks

6. Initialize monorepo structure: `/backend` (FastAPI + LangChain/LangGraph pipeline), `/ingestion` (corpus pull + indexing scripts), `/eval` (gold set + harness), `/frontend` (optional, Next.js), `/infra` (Docker Compose, deployment configs), `/docs`.
7. Set up `pyproject.toml`/`requirements.txt` with pinned versions, pre-commit hooks (ruff/black for Python, eslint/prettier if frontend), and a `.env.example` listing every required secret (no real secrets committed — verify `.gitignore` covers `.env`, `*.key`, credential JSON files).
8. Define core Pydantic data contracts up front and treat them as the interface every phase must honor:

```
# backend/app/schemas.py (skeleton — agent fills in field-level validation)

class ClaimElement(BaseModel):
    element_id: str
    text: str
    element_type: Literal["structural", "functional", "chemical", "numeric", "method_step"]

class DecomposedClaim(BaseModel):
    raw_claim_text: str
    claim_number: Optional[str]
    elements: list[ClaimElement]

class RetrievedDocument(BaseModel):
    doc_id: str            # patent publication number
    title: str
    snippet: str
    retrieval_sources: list[Literal["bm25", "dense", "hyde"]]
    raw_scores: dict[str, float]      # per-source raw score
    fused_score: float                # RRF output
    rerank_score: Optional[float]
    matched_elements: list[str]       # which ClaimElement ids this doc addresses
    explanation: Optional[str]        # per-result relevance explanation

class PriorArtSearchResponse(BaseModel):
    query_claim: DecomposedClaim
    results: list[RetrievedDocument]
    pipeline_stage_counts: dict[str, int]   # how many candidates survived each stage
    disclaimer: str
    latency_ms: dict[str, float]            # per-stage timing
```

9. Set up Docker Compose skeleton with placeholder services: `backend`, `qdrant`, (optional) `frontend`, (optional) `redis` for caching — all should build and start (even with stub endpoints) before Phase 1 begins.
10. Set up CI skeleton (GitHub Actions) that runs lint + a trivial smoke test on every push, even before real tests exist — the pipeline itself should be proven working early, not bolted on in Phase 7.

Definition of Done

Definition of Done — criterion	How it is verified
`docker compose up` starts all defined services without error on a clean machine	Manual run on a fresh clone; capture as a recorded terminal log kept in `/docs/verification/`
All Pydantic schemas defined and importable with no circular-import errors	`python -c "from backend.app import schemas"` exits 0
CI pipeline is green on an empty/stub commit	GitHub Actions run visible and passing
.env.example lists every secret needed by later phases (API keys for embeddings, rerank, LLM, Qdrant)	Manual review checklist cross-referenced against Section 1.4 tech choices

PHASE 1

Corpus Acquisition & Indexing

Goal
Pull a bounded, real patent corpus and build both the BM25 lexical index and the dense vector index against it — the foundation every retrieval layer depends on.

Estimated effort: 2–3 days

Tasks

11. Write `/ingestion/pull_corpus.py`: connects to the chosen data source (Section 1.1), pulls the bounded CPC-scoped slice (Section 1.2), and normalizes into a canonical `PatentDocument` record (id, title, abstract, claims[], CPC codes, publication date, cited-by/cites lists for later eval-set use).
12. Persist normalized corpus to a local Parquet/JSONL store (`/data/corpus/`) as the single source of truth — both indexes are built from this, not from re-hitting the external API each time, so indexing is reproducible and rate-limit-safe.
13. Write `/ingestion/build_bm25_index.py`: tokenizes claim + abstract text (patent-aware tokenization — preserve chemical formulae, patent numbers, and hyphenated compound terms as meaningful tokens rather than destroying them with naive stemming), builds the `rank-bm25` index, and serializes it.
14. Write `/ingestion/build_dense_index.py`: chunks each document appropriately (claim-level and abstract-level chunks, not the whole document as one vector — claim-level granularity is what makes per-element matching possible in Phase 3), embeds with the selected model, and upserts into Qdrant with payload metadata (CPC codes, publication date, patent id) so results can be filtered later.
15. Build the citation-graph extraction needed for Section 1.3's weak-label eval set: for each patent with cited prior art in the corpus, record the citation pairs into `/eval/citation_pairs.json`.

Definition of Done

Definition of Done — criterion	How it is verified
Corpus of 50k–150k real patent documents pulled and normalized	`wc -l` / row count on `/data/corpus/` matches target; spot-check 5 random records against the original source manually
BM25 index returns non-empty, sensible results for a known exact-match query (e.g. a real patent number or chemical formula present in the corpus)	Scripted smoke test: query with a known exact string, assert the source document appears in top-3
Dense index returns non-empty, sensible results for a paraphrased query	Scripted smoke test: query with a semantic paraphrase of a known document's content, assert it appears in top-10
Both indexes persist and reload without rebuilding from scratch	Restart Qdrant container and BM25 pickle load; confirm same query returns same results
Citation-pair eval seed file exists with ≥ 200 real citation pairs	Row count check on `/eval/citation_pairs.json`

PHASE 2

Query Understanding — Decomposition & HyDE

Goal
Turn a raw compound patent claim into (a) atomic searchable elements and (b) hypothetical-document expansions, both of which feed the retrieval layer in Phase 3.

Estimated effort: 2 days

Tasks

16. Build the claim decomposition chain (LangChain/LangGraph node) that takes raw claim text and outputs a `DecomposedClaim` (per the Phase 0 schema): break a compound claim ("A device comprising X, wherein X is configured to Y, and further comprising Z...") into atomic `ClaimElement`s, each tagged with a type (structural / functional / chemical / numeric / method_step).
17. Prompt-engineer this as a structured-output call (function calling / JSON mode against whichever provider is active) rather than free-text parsing — the schema in Phase 0 is the contract, and structured output enforcement avoids brittle regex parsing of LLM prose.
18. Build the HyDE step: for each atomic element (or for the claim as a whole — test both, see below), prompt the LLM to generate a hypothetical prior-art passage that *would* satisfy/anticipate that element, then embed that hypothetical passage instead of (or alongside) the raw element text for the dense-retrieval step. This is what closes the vocabulary gap between formal claim language and how the same concept is actually described in a different patent's specification.
19. Make the decomposition-granularity choice (element-level HyDE vs whole-claim HyDE) an explicit, logged decision with a comment explaining the tradeoff — element-level gives finer per-element retrieval but costs more LLM calls and adds latency; this tradeoff should be visible in the Phase 5 lift table (query-transform row), not just asserted in a comment.
20. Add a validation/repair step: if the LLM's decomposition output fails schema validation (malformed JSON, missing required fields), retry once with an error-correction prompt before failing the pipeline stage — this is the first of several "fail safely, not silently" points from Section 0.5.

Definition of Done

Definition of Done — criterion	How it is verified
Given a real multi-element claim, decomposition produces ≥ 3 correctly-typed atomic elements	Unit test with 5 hand-picked real claims of varying complexity, asserting element count and spot-checked types
HyDE generation produces a plausible hypothetical passage per element (not empty, not a refusal, not a restatement of the element itself)	Manual review of 10 generated HyDE passages against a rubric (does it read like real patent prose? does it add new phrasing vs the input?)
Malformed LLM output triggers the retry-then-typed-error path, never a raw crash	Unit test that mocks a malformed LLM response and asserts graceful handling

Definition of Done — criterion	How it is verified
Full decomposition+HyDE step for one claim completes in a bounded, logged time	Timing captured in `latency_ms` field of the response schema; assert < a documented threshold (e.g. 8s) in test

PHASE 3

Hybrid Retrieval, Fusion & Reranking Pipeline

Goal
Wire BM25 + dense (+ HyDE-expanded dense) retrieval, fuse with RRF, and apply the cross-encoder rerank — the actual multi-stage pipeline the whole project exists to prove out.

Estimated effort: 3–4 days

Tasks

21. Build the retriever adapter interface so BM25, dense, and (Cohere / BGE) rerank are swappable behind one contract — this both makes the Phase 3.5 stretch options (SPLADE, alternate rerankers) trivial to add, and is a clean engineering pattern to describe in an interview.

```
# backend/app/retrieval/base.py

class Retriever(Protocol):
    def search(self, query: str, k: int, filters: dict | None = None) -> list[ScoredDoc]: ...

class Reranker(Protocol):
    def rerank(self, query: str, candidates: list[ScoredDoc], top_n: int) -> list[ScoredDoc]: ...

# Concrete: BM25Retriever, DenseRetriever, HydeDenseRetriever
#           CohereReranker, BgeCrossEncoderReranker
```

22. Run retrieval per claim element (and optionally the whole-claim query too) across BM25 and dense retrievers in parallel (async), each returning top-N candidates with raw scores.
23. Implement Reciprocal Rank Fusion from scratch: for each document, `RRF_score = sum(1 / (k_rrf + rank_in_list_i))` across all lists it appears in, `k_rrf` a small constant (commonly 60) — the key property to preserve and be able to explain is that this needs no tunable weight between BM25 and dense, unlike a naive linear score blend.
24. Take the fused, deduplicated candidate set (dedupe by patent id, merging matched-element provenance) and pass the top-N (e.g. top 50–100) through the cross-encoder reranker (Cohere Rerank API or self-hosted BGE cross-encoder) to produce the final rerank score and ordering.
25. Aggregate per-document matched elements: track, across the whole pipeline, which of the original claim's atomic elements each surviving candidate document actually addresses — this is required input for Phase 4's explanation generation and is part of the reason element-level retrieval (not whole-claim-only) was chosen in Phase 2.
26. Wire the whole thing as a LangGraph graph (decompose → HyDE → parallel retrieve → fuse → rerank → explain) with explicit per-node error handling: a failed rerank call falls back to RRF-only ordering with a logged warning, not a hard failure of the whole request — second instance of the "fail safely" principle.

Definition of Done

Definition of Done — criterion	How it is verified
BM25-only retrieval surfaces a known exact-match fixture document (patent number/chemical formula) that dense-only retrieval fails to surface in top-10	Integration test explicitly constructed to reproduce the brief's forcing-function failure mode — this test is the single most important test in the whole project
Dense-only retrieval surfaces a known semantic-paraphrase fixture document that BM25-only fails to surface in top-10	Mirror integration test for the opposite failure mode
RRF-fused results contain both fixture documents from the two tests above in top-10, where at least one single-method retriever missed one of them	Integration test asserting the fusion recovers what either method alone drops
Reranker changes result ordering (not a no-op) on a query where relevance intuitively disagrees with raw fused rank	Test with a candidate set where a lower-RRF-ranked-but-more-relevant doc should move up post-rerank; assert the reorder happens
A rerank API failure (mocked timeout/error) degrades to RRF ordering with a logged warning, not a pipeline crash	Unit test mocking reranker exception
Full pipeline (real claim in → ranked results out) completes end-to-end for 10 varied test claims with no unhandled exceptions	Scripted batch run over `/eval/gold_set.json` claims, asserting 0 exceptions and non-empty results for each

PHASE 4

Explanations, API Surface & UI

Goal
Generate per-result relevance explanations, expose a clean documented API, and (optionally) a UI — the layer that turns ranked doc IDs into something a human can actually evaluate.

Estimated effort: 2–3 days (API) + 2–3 days (UI, optional)

Tasks — explanations

27. For each final top-k result, generate a short, grounded explanation: which claim element(s) it matches, via which retrieval path(s) (BM25 exact-match / dense-semantic / HyDE-expanded), and a one-line rationale — built from the `matched_elements` and `retrieval_sources` data already tracked in Phase 3, plus one LLM call to phrase it as readable prose grounded strictly in that structured data (not a free-floating LLM claim about the document's relevance).
28. Explicitly prevent explanation hallucination: the explanation prompt must only reference elements/snippets actually present in the retrieved document's indexed text and the claim's decomposed elements — pass these as the only grounding context, and spot-check for fabricated specifics in Phase 5.5's faithfulness eval.

Tasks — API

29. Expose FastAPI endpoints: `POST /search` (raw claim text in, `PriorArtSearchResponse` out), `GET /health`, `GET /ready` (checks Qdrant + downstream API connectivity, not just process liveness), `GET /eval/latest` (serves the current lift table, see Phase 5).
30. Auto-generated OpenAPI/Swagger docs (FastAPI default) reviewed for clarity — this doubles as Phase 8 documentation and as a reviewer-friendly "try it yourself" surface.
31. Ensure the disclaimer text from Section 1.5 is a non-optional field on every `/search` response, not just a UI-layer addition — an API consumer bypassing the UI must still receive it.

Tasks — UI (optional but recommended for resume impact)

32. Build a minimal Next.js page: claim text input → decomposed elements shown → ranked results with per-result explanation, matched-element badges, and retrieval-source badges (BM25/dense/HyDE icons) — the visual proof of the multi-stage pipeline, which is what makes this stand out over a plain JSON API in a portfolio demo.
33. Render the Section 1.5 disclaimer prominently and persistently (not a dismissible one-time modal) — required, not optional, per Section 0.4.
34. Add a small "how this result was found" expandable detail per result showing the raw BM25 rank, dense rank, RRF score, and rerank score side by side — this is the feature that makes the lift-table story tangible to a non-technical viewer of the demo, and is worth the build time.

Definition of Done

Definition of Done — criterion	How it is verified
Every result in a real end-to-end run has a non-empty, specific (not generic/templated) explanation	Manual review of 15 results across 3 different claims for explanation specificity
Explanation faithfulness spot-check: no explanation references a claim element or document detail that isn't actually present in the underlying data	Manual review against Phase 3's matched_elements ground truth for 15 sampled results; formalized further in Phase 5.5 with RAGAS faithfulness metric
`/health` returns 200 always when process is up; `/ready` returns 503 when Qdrant is intentionally stopped	Test: stop Qdrant container, curl `/ready`, assert 503; restart, assert 200
Disclaimer field present and non-empty on every `/search` response, verified by schema, not just eyeballing	Pydantic field marked required (non-Optional); test asserts presence
Swagger UI at `/docs` loads and every endpoint is callable from it with a real example	Manual click-through, screenshot captured for `/docs/verification/`
(If UI built) Full user flow — paste claim, see decomposition, see ranked results with explanations and source badges — works with no console errors	Manual browser walkthrough + browser console check

PHASE 5

Evaluation Harness & the Lift Table

Goal
Build the precision/recall/MRR measurement harness against the Phase 1 gold set and produce the layer-by-layer lift table that is the entire evidentiary basis for this project's engineering claims.

Estimated effort: 2–3 days

5.1 Metrics to implement

Metric	Definition (k = cutoff, typically 5/10/20)	Why it matters here
Precision@k	Relevant docs in top-k ÷ k	Directly answers "of what I showed the attorney, how much was actually useful"
Recall@k	Relevant docs in top-k ÷ total relevant docs	The legal-risk metric — recall failures are the missed-invalidating-document liability from the brief's forcing function
MRR (Mean Reciprocal Rank)	Average of 1/rank of first relevant doc, across queries	Captures "how far down did I have to scroll to find anything useful"
nDCG@k	Rank-position-discounted relevance gain, normalized	Rewards getting the most relevant doc near the top, not just present somewhere in top-k

5.2 The four pipeline configurations to measure

This is the exact structure the brief asks for — build the harness to run all four automatically over the gold set, not as one-off manual runs:

35. **Baseline:** BM25-only, no query transform, no rerank — the naive lexical-search floor.
36. **+ Hybrid:** BM25 + dense retrieval, RRF-fused, still no rerank.
37. **+ Rerank:** Hybrid + fusion, now with the cross-encoder rerank pass applied to the fused candidates.
38. **+ Query-transform:** Full pipeline — adds claim decomposition + HyDE expansion on top of hybrid+rerank.

5.3 Build the harness

39. Write `/eval/run_lift_table.py`: loads `/eval/gold_set.json`, runs all 40+ held-out claims through each of the four configurations above, computes all four metrics at k ∈{5, 10, 20} per configuration, and writes results to `/eval/results/lift_table_<timestamp>.json` plus a rendered Markdown table.
40. Make this harness re-runnable as a CLI command and as a CI job (Phase 7) so the lift table is never a stale, hand-typed claim — it is regenerated from the actual current pipeline on demand.
41. Include, alongside the metrics table, at least 2 concrete worked examples in the eval report: one claim where BM25-only genuinely missed an exact-match invalidating document that hybrid caught, and one where dense-only missed a paraphrase that hybrid caught — numbers alone don't tell the forcing-function story as well as a named example.

42. Compute and report statistical context, not just point estimates: with ~40 queries, note the metric spread (min/max/stdev across queries) so the lift table doesn't overstate precision on a small eval set — this level of honesty is itself a strong engineering signal in review.

5.4 Expected (not guaranteed) shape of results — and what to do if reality differs

> **Important: do not force the numbers**
> The likely pattern is recall improving substantially from Baseline → Hybrid (BM25 alone misses paraphrases), precision improving further from Hybrid → +Rerank (reordering pushes true positives up), and +Query-transform giving the biggest lift specifically on claims with heavy technical paraphrase or acronym drift. If your actual measured results don't show a clean monotonic improvement at every step — for instance if HyDE adds latency without a measurable recall gain on your specific corpus — report that honestly in the README. An honest "this layer didn't earn its place on my eval set, and here's the likely reason" is a stronger engineering artifact than a suspiciously perfect staircase of numbers, and it's exactly what the brief's forcing-function framing rewards.

5.5 Stretch: RAGAS layer for explanation quality

Once retrieval-layer metrics are solid, optionally layer RAGAS (or a custom equivalent) on top to score the Phase 4 explanations for faithfulness (does the explanation only state things actually supported by the retrieved document?) and relevance (does the explanation address the actual claim element it's attached to?). This is a distinct axis from retrieval quality — a well-retrieved document can still get a bad explanation — and is worth calling out separately in the eval report rather than folding into the retrieval lift table.

Definition of Done

Definition of Done — criterion	How it is verified
Lift table generated and saved, covering all 4 configurations × all 4 metrics × k∈{5,10,20}, over the full held-out gold set	`/eval/results/lift_table_<latest>.json` exists and row/column count matches spec; re-run twice, confirm results are stable (not randomly different each run)
At least 2 named worked examples included in the eval report illustrating the forcing function concretely	Manual review of `/eval/results/` Markdown report
Harness is a single CLI command runnable by a stranger with the repo cloned and services running	Fresh-clone dry run of `python eval/run_lift_table.py`
Metric spread/variance reported alongside point estimates, not just averages	Manual review of report format
`GET /eval/latest` API endpoint (Phase 4) serves this same data	Integration test hitting the endpoint and validating schema match with the on-disk report

PHASE 6

Hardening — Security, Reliability, Observability

Goal
Everything that separates a working demo from something that survives contact with real traffic, bad input, and abuse.

Estimated effort: 2–3 days

6.1 Input handling & abuse resistance

43. Enforce a maximum claim-text length on `/search` (patent claims are long but bounded — pick a generous but finite cap, e.g. 10,000 characters) and return a typed 422 error, not a silent truncation or an OOM, for anything larger.
44. Rate-limit `/search` per API key/IP (e.g. token-bucket via `slowapi` or an API gateway) — every downstream call (embedding, HyDE LLM call, rerank API) costs real money per request, so this endpoint is a real cost-abuse surface, not a theoretical one.
45. Require an API key for non-health endpoints (simple header-based key check is sufficient for a portfolio deployment; document that a production system would use a real auth provider).
46. Sanitize/validate that claim text isn't being used to inject instructions into the decomposition or explanation LLM prompts — basic prompt-injection awareness (e.g. clearly delimited user content in prompts, not naive string concatenation) since this pipeline does pass user-supplied text into multiple LLM calls.

6.2 Reliability

47. Timeouts on every external call (embedding API, rerank API, LLM calls, Qdrant queries) with sane defaults, and the graceful-degradation paths from Phase 2/3 (schema-retry, rerank-fallback-to-RRF) covered by explicit tests here, not just implemented and hoped-for.
48. Retry with backoff on transient failures (5xx, timeout) for external API calls, capped at a small retry count so a downstream outage degrades the request, not hangs it indefinitely.
49. Idempotent, safe re-indexing: re-running the Phase 1 ingestion/indexing scripts on already-indexed data should not duplicate documents or corrupt the index — test this explicitly, since corpus updates are a realistic future need.

6.3 Observability

50. Structured JSON logging (not print statements) for every pipeline stage, including per-stage latency and candidate-count-survived (already partially captured in the Phase 0 schema's `latency_ms` and Phase 3's stage counts) — emit these as real log lines, not just response fields.
51. Basic request tracing: a request ID generated at the API boundary, threaded through every pipeline stage's logs, so a single slow or failed request can be reconstructed from logs end-to-end.
52. A minimal metrics surface (even a simple `/metrics` Prometheus-format endpoint, or just structured logs aggregable later) covering request count, error rate, and per-stage latency distribution — doesn't need a full Grafana dashboard for a portfolio project, but the data needs to exist.

Definition of Done

Definition of Done — criterion	How it is verified
Oversized claim input returns typed 422, does not crash or hang the process	Test with a >10k-char input; assert response code and shape
Rate limit triggers correctly under rapid repeated requests and returns 429	Test issuing requests above the configured threshold in a tight loop
Missing/invalid API key returns 401/403 on protected endpoints	Test with no key and with a garbage key
Simulated downstream timeout (mocked slow embedding/rerank call) triggers configured timeout + fallback path, not an indefinitely hanging request	Test with a mocked slow dependency and an asserted max wall-clock bound on the response
Re-running ingestion on already-indexed data produces no duplicate documents in Qdrant or BM25 index	Run ingestion twice, assert document count unchanged after 2nd run
Every request is traceable end-to-end via a single request ID across all logged pipeline stages	Manual log review: trigger one request, grep logs by request ID, confirm every stage present

PHASE 7

Self-Test Gate & Deployment

Goal
This is the explicit "test itself, come out error-free, then deploy itself" phase from the brief — an automated gate that must pass before deployment proceeds, followed by the actual deployment.

Estimated effort: 1–2 days

7.1 The self-test gate — what "error-free" concretely means here

"Fully error-free" is not a testable absolute claim for any real software system — what is testable, and what this plan commits to instead, is a defined, automated gate that must be green before deployment is allowed to proceed. Build this as a single CI job (`ci_full_gate.yml`) that runs, in order, and fails the build (blocking deployment) if any step fails:

53. Lint + type-check (ruff/mypy for Python, eslint/tsc for frontend if built) — zero errors, warnings allowed but logged.
54. Full unit test suite (all Definition-of-Done unit tests from Phases 0–6) — 100% pass required, no skipped tests without an explicit, reviewed `# TODO` justification comment.
55. Full integration test suite, including the two forcing-function tests from Phase 3 (BM25-catches-what-dense-misses, dense-catches-what-BM25-misses) and the end-to-end 10-claim batch run — 100% pass required.
56. Fresh-environment build check: `docker compose build && docker compose up -d` on a clean runner, then a smoke-test script hitting `/health`, `/ready`, and one real `/search` call end-to-end — must succeed within a bounded startup time.
57. Lift-table regeneration (Phase 5 harness) runs and the resulting metrics are compared against a stored baseline snapshot — fail the gate (or at minimum flag loudly, reviewer's call) if precision/recall regresses beyond a documented tolerance from the last known-good run, so a code change can't silently degrade retrieval quality without anyone noticing.
58. Security basics: dependency vulnerability scan (`pip-audit` / `npm audit`), and a check that no secrets are present in the committed codebase (`gitleaks` or equivalent).

7.2 Deployment path — pick based on what you actually need

The brief asks the system to "deploy itself properly with full functioning." Interpreted honestly: a fully unattended, zero-human-decision deployment isn't a realistic or even desirable target for a first production deployment of a project like this (a human should approve a deploy, at minimum once). What is realistic and matches the spirit of the ask is a one-command, gate-enforced deployment pipeline. Choose the path that matches what you actually want at the end:

If you want…	Do this	Effort
A live, link-in-resume demo URL	Backend (FastAPI + Qdrant) on Railway/Render/Fly.io; frontend (if built) on Vercel; CI job auto-deploys on merge to `main` only after the Section 7.1 gate is green	0.5–1 day, small ongoing hosting cost (or free-tier-eligible for a portfolio-scale corpus)

If you want…	Do this	Effort
A fully local, runs-on-my-machine reproducible system (no hosting cost)	Single `docker compose up` after cloning; document this clearly as the "deployment" in the README, with the same CI gate enforced on every push even without a live URL	Near-zero additional effort beyond Phase 0's Compose setup
A single persistent cloud VM (more control, still cheap)	One small VM (e.g. a $5–$10/mo instance) running Docker Compose, with the CI gate triggering a `docker compose pull && up -d` over SSH on merge to `main`	0.5 day, small fixed monthly cost

Recommendation for a resume-facing project

Option 1 (managed platform demo URL) gives the best return for a resume/portfolio use case — a reviewer can click a link and see it work with zero setup. Keep the corpus and index size within that platform's free/cheap tier limits (Section 1.2's bounded-corpus decision was made with exactly this in mind). Keep Option 2 always available as the fallback path documented in the README regardless of which you deploy live, since free-tier hosting can lapse.

Definition of Done

Definition of Done — criterion	How it is verified
Section 7.1 gate exists as a single CI workflow and is green on the commit being deployed	GitHub Actions run link captured in `/docs/verification/`
Deployment (whichever path chosen) is triggered only after the gate passes — verified by CI config, not just described in prose	Review `.github/workflows/` — deploy job has `needs:` dependency on the gate job
Live system (if deployed) responds correctly to `/health`, `/ready`, and a real `/search` call, from a machine outside the deployment environment	curl/Postman test from a separate machine/network post-deploy
Lift-table regression check is part of the gate and has a documented tolerance threshold	Review CI config + `/docs/eval_regression_policy.md`
A rollback path is documented (even if manual) — how to revert to the last known-good deployed version	README section reviewed for presence and clarity

PHASE 8

Documentation & Resume Packaging

Goal
The artifact a hiring manager, interviewer, or your future self actually reads — this phase is not optional polish, it's where the engineering work becomes legible to someone who didn't build it.

Estimated effort: 1 day

8.1 README structure (in this order)

59. One-paragraph summary + architecture diagram (decompose → HyDE → hybrid retrieve → RRF fuse → rerank → explain) — a picture here does more work than any amount of prose.
60. The forcing function, stated explicitly, with the two concrete worked examples from Phase 5.3 (the specific claim/document pairs where BM25-only or dense-only failed and hybrid caught it) — this is the single most interview-relevant paragraph in the whole README.
61. The lift table itself, rendered as a Markdown table, pulled live from `/eval/results/` — not hand-typed, so it can't silently drift out of sync with the actual measured system.
62. Setup instructions: both the one-command Docker path and a from-scratch manual path, tested by literally following your own instructions on a clean environment before publishing.
63. Tech stack table with honest alternatives noted (mirroring Section 1.4) — shows deliberate choice-making, not just "I used whatever tutorial I found."
64. Known limitations section, stated plainly: bounded corpus size and technology area, weak-label eval-set caveats, the standing legal disclaimer restated, and any layer from the lift table that didn't measurably earn its place (per Section 5.4) — candor here reads as engineering maturity, not weakness.
65. Link to the live demo (if deployed) and to the CI gate / latest passing run, so a reviewer can independently verify "error-free" rather than take it on faith.

8.2 Resume-line drafting guidance

Do not describe this project on a resume with unfalsifiable adjectives ("advanced," "cutting-edge," "robust") — use the actual measured numbers from your own Phase 5 lift table, since they're now real and yours. A defensible pattern:

```
Built a multi-stage patent prior-art retrieval system (LangChain/LangGraph,
Qdrant, Cohere Rerank) combining BM25 + dense hybrid search with Reciprocal
Rank Fusion and cross-encoder reranking; measured a [X]% recall@10 and [Y]%
precision@10 improvement over BM25-only baseline on a held-out, citation-
labeled evaluation set of 40+ real patent claims.
```

Fill in [X] and [Y] with your own Phase 5 numbers — do not estimate or round up. If a reviewer asks about the numbers in an interview, being able to explain exactly how they were measured (Section 5.1–5.3) and where the eval set's limitations are (Section 5.4, 1.3) is worth far more than the numbers being impressively large.

Definition of Done

Definition of Done — criterion	How it is verified
README followed literally by someone unfamiliar with the project results in a running system	Ideally: have another person (or a fresh agent session with no prior context) attempt setup from the README alone and report friction points
Lift table in README is programmatically generated/embedded, not manually retyped	Check README build/update process — script or CI step that regenerates the embedded table
Limitations section present and specific, not generic boilerplate	Manual review against Section 8.1 checklist items
Resume-line draft uses only measured numbers with no unfalsifiable superlatives	Manual review against Section 8.2 guidance

2. Phase Summary & Suggested Timeline

Total realistic solo build time: roughly 3–4 weeks at a sustainable pace (evenings/weekends), or 10–14 focused full days. Phases 0–3 are strictly sequential (each depends on the last); Phase 4's UI track can run in parallel with Phase 5's eval-harness track once Phase 3 is stable, since neither depends on the other.

Phase	Name	Depends on	Effort	Parallelizable?
0	Repo, environment, contracts	—	0.5–1 day	No — do first
1	Corpus acquisition & indexing	Phase 0	2–3 days	No
2	Decomposition & HyDE	Phase 0	2 days	No
3	Hybrid retrieval, fusion, rerank	Phases 1, 2	3–4 days	No
4	Explanations, API, UI	Phase 3	2–3 days (+2–3 UI)	Yes — parallel with Phase 5
5	Eval harness & lift table	Phase 3	2–3 days	Yes — parallel with Phase 4
6	Hardening (security/reliability/observability)	Phases 3, 4	2–3 days	No
7	Self-test gate & deployment	Phases 5, 6	1–2 days	No
8	Documentation & resume packaging	Phase 7	1 day	No — do last

3. Ready-to-Paste Prompts for Your IDE Agent

Paste one phase at a time into your IDE agent (Claude Code, Cursor, Windsurf, Copilot Workspace, etc.), in order. Do not paste all phases at once — the whole design of this plan is sequential, gated execution, and an agent working from all phases simultaneously is more likely to skip a Definition-of-Done check. Each prompt below references "the attached build specification," meaning this document — attach or paste the relevant phase section alongside it.

Phase 0

```
Set up the repository skeleton exactly as specified in Phase 0 of the attached build specification (monorepo structure, Pydantic contracts, Docker Compose skeleton, CI skeleton). Do not write any retrieval logic yet. When done, run through every item in the Phase 0 Definition of Done table and report the result of each check explicitly — pass or fail — before stopping.
```

Phase 1

```
Implement Phase 1 of the attached build specification: corpus acquisition and indexing. Use the data sources and bounding decisions specified in Section 1.1–1.2 exactly — do not substitute a synthetic or invented corpus. Build both the BM25 and dense indexes as specified. When done, run through every item in the Phase 1 Definition of Done table, show me the actual output/evidence for each (not just a claim that it passed), and stop for my review before proceeding to Phase 2.
```

Phase 2

```
Implement Phase 2 of the attached build specification: claim decomposition and HyDE expansion, using the Phase 0 schemas as the fixed contract. Enforce structured LLM output (not free-text parsing) as specified. When done, run every item in the Phase 2 Definition of Done table with real test claims and show me the actual decomposition + HyDE output for at least 3 example claims before proceeding.
```

Phase 3

```
Implement Phase 3 of the attached build specification: hybrid retrieval, RRF fusion, and cross-encoder reranking, using the adapter interface pattern specified. This phase's two forcing-function integration tests (BM25 catches what dense misses, and vice versa) are the most important tests in this project — build them first, watch them demonstrate the failure mode on your actual corpus and indexes before you build the fix, then confirm fusion resolves both. Show me the actual before/after results for both tests, not just pass/fail status.
```

Phase 4

```
Implement Phase 4 of the attached build specification: per-result explanation generation and the FastAPI surface (build the optional Next.js UI too if I've asked for it). Explanations must be grounded strictly in the matched_elements/retrieval_sources data already computed in Phase 3 — do not let the explanation LLM call invent relevance claims not backed by that structured data. The disclaimer field specified in Section 1.5 must be a required, non-optional field on every response. Show me 5 real end-to-end example results with their explanations for review.
```

Phase 5

```
Implement Phase 5 of the attached build specification: the evaluation harness and lift table, covering all 4 pipeline configurations and all 4 metrics at k∈{5,10,20} specified in Section 5.1–5.3, run against the real held-out gold set (not a placeholder). Report the actual measured numbers to me — if any layer doesn't show improvement, tell me that honestly per Section 5.4's guidance rather than adjusting the eval to make it look better. Include the 2 worked examples specified in 5.3.
```

Phase 6

```
Implement Phase 6 of the attached build specification: security, reliability, and observability
hardening. Every item in the Phase 6 Definition of Done table must have an actual passing test, not
just an implemented feature. Show me the test results for each.
```

Phase 7

```
Implement Phase 7 of the attached build specification: the full self-test CI gate and the deployment
path I've chosen from Section 7.2. The gate must actually block deployment on failure — show me the
CI configuration proving the deploy job depends on the gate job passing. After deployment, run a real
request against the live system from outside the deployment environment and show me the result.
```

Phase 8

```
Complete Phase 8 of the attached build specification: README and documentation. Follow Section 8.1's
structure exactly, and use only the real measured numbers from my actual Phase 5 lift table results
(do not invent, round up, or estimate numbers). Draft the resume line per Section 8.2 using my real
numbers, and show me the final README for review before we're done.
```

A. Appendix A — Risk Register

Risk	Likelihood	Mitigation (already built into the plan above)
Chosen embedding/LLM/rerank model becomes unavailable or renamed between plan-writing and build time	Medium	Section 1.4 deliberately avoids hardcoding model version strings; Phase 2/3 adapter interfaces make provider swaps a config change, not a rewrite
Free-tier data source (BigQuery public dataset, USPTO API) rate-limits or changes terms	Medium	Section 1.1 lists 3–4 alternative sources; Section 1.2's bounded-corpus approach minimizes total pull volume needed from any one source
Eval set is too small / weak-labeled to produce statistically meaningful lift-table numbers	Medium–High	Section 5.3 mandates reporting variance alongside point estimates, and Section 5.4 explicitly instructs honest reporting over forced-clean numbers — this risk is disclosed, not hidden
A retrieval layer (e.g. HyDE) doesn't measurably improve metrics on the specific chosen corpus	Medium	This is treated as a valid, reportable outcome throughout (Section 5.4, 8.1) rather than a build failure — the forcing-function framing survives either way
Free/cheap hosting tier is insufficient for corpus+index size at deploy time	Low–Medium	Section 1.2's corpus bound (50k–150k docs) and Section 7.2's tiered deployment options are sized with this constraint in mind from the start
Solo build timeline slips well past the Section 2 estimate	Medium	Phase 4/5 parallelization noted explicitly; Phases can be paused after any Definition-of-Done gate with a working, demoable system at each checkpoint — no phase leaves the system in a broken intermediate state

B. Appendix B — Glossary (for README reuse)

Term	Plain-language definition
Prior art	Any publicly available evidence (patents, publications, products) that existed before a claim's filing date and is relevant to whether the claim is new/non-obvious
Invalidating prior art	Prior art that anticipates (matches every element of) or renders obvious a specific patent claim, potentially invalidating it
Independent claim	A patent claim that stands alone (doesn't reference another claim); this system operates on independent claims since they contain the full set of limitations to search against
BM25	A classic lexical/keyword ranking algorithm scoring documents by term frequency, adjusted for document length — excels at exact-token matches
Dense retrieval	Search using vector embeddings to find semantically similar text even without shared exact wording
HyDE (Hypothetical Document Embeddings)	A technique where an LLM first generates a hypothetical answer/passage to the query, and that hypothetical text is embedded and searched, rather than embedding the raw query — helps close vocabulary gaps
Reciprocal Rank Fusion (RRF)	A method for merging multiple ranked lists into one, using only each document's rank position (not raw scores) in each list — requires no tuning of relative weight between lists
Cross-encoder reranker	A model that scores a (query, document) pair jointly (more accurate, more expensive) as a final refinement pass over a smaller candidate set, versus retrieval models that score each independently
Precision@k / Recall@k	Precision@k: fraction of the top-k results that are actually relevant. Recall@k: fraction of all relevant documents that were found within the top-k
Lift table	A table comparing a metric (e.g. Recall@10) across progressively more sophisticated pipeline configurations, showing the measured improvement ("lift") each added layer provides

End of build specification.